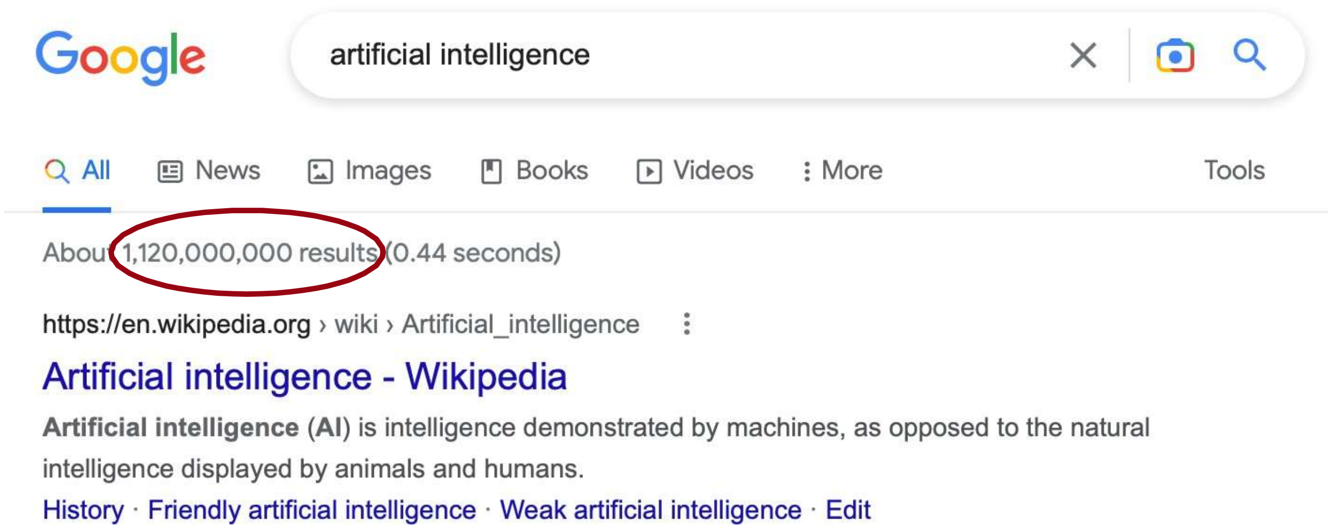
Natural Language Processing

Week 3: Part-of-speech Tagging

Dr.Shahed Almobydeen

Last time: Information retrieval



What if you are interested only in **a very specific question** on AI?

Learning outcomes

- **Application**: Information extraction
- **NLP concepts and techniques**:
    - *part-of-speech tagging* (this week)
    - *sequence modelling* (this week)
    - *chunking* and *parsing* (next week)
- **Practical skills**: you will learn how to use further NLP tools and toolkits, and by the end of these two weeks, you will build your own end-to-end information extraction application

This week

01 Introduction to Information Extraction (IE)

02 Introduction to PoS tagging

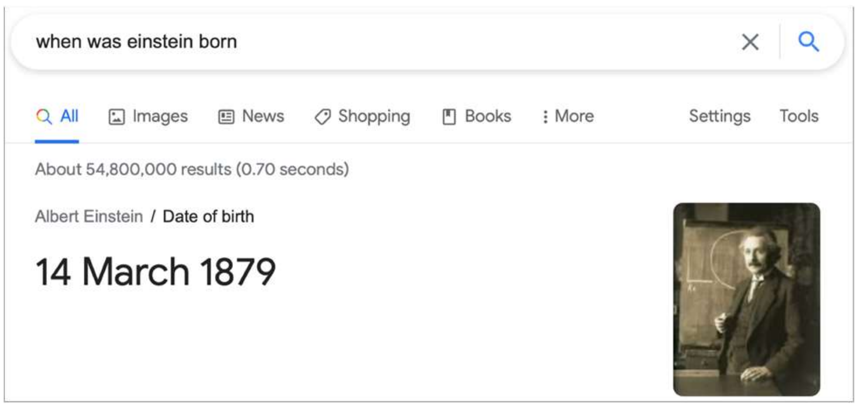
03 Sequence modelling and labelling in NLP

04 PoS tagging with HMMs

Introduction to Information Extraction (IE)

Why Information Extraction?

- **Information Extraction** (**IE**) is concerned with identification and extraction of structured information (particular facts) from an unstructured or only partially structured input
- **Example**: suppose you want to know "**When** was Einstein born?" or "**Where** was Einstein born?"
- Total number of pages about Einstein – over 135 mln



Example

Albert Einstein was born in Ulm, in the Kingdom of Wurttemberg in the German Empire, on 14 March 1879

Example

Albert Einstein was born in Ulm, in the Kingdom of Wurttemberg in the German Empire, on 14 March 1879

Who?
(subject)

Example

Albert Einstein was born in Ulm, in the Kingdom of Wurttemberg in the German Empire, on 14 March 1879

Did what?
(action)

Example

Albert Einstein was born in Ulm, in the Kingdom of Wurttemberg in the German Empire, on 14 March 1879

Where?
(location)

Where?
(location)

Where?
(location)

Example

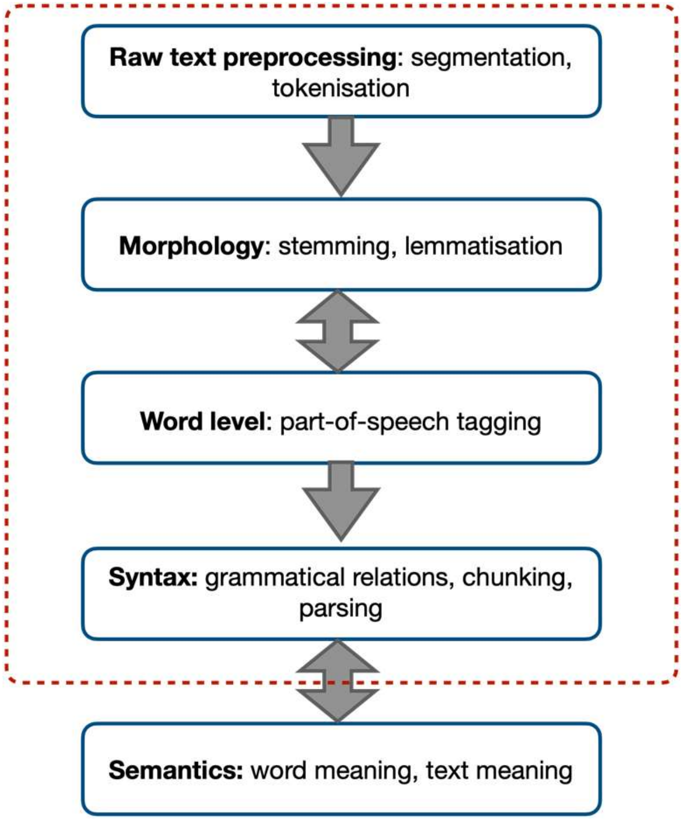
Albert Einstein was born in Ulm, in the Kingdom of Wurttemberg in the German Empire, on 14 March 1879

When?
(time reference)

Challenges

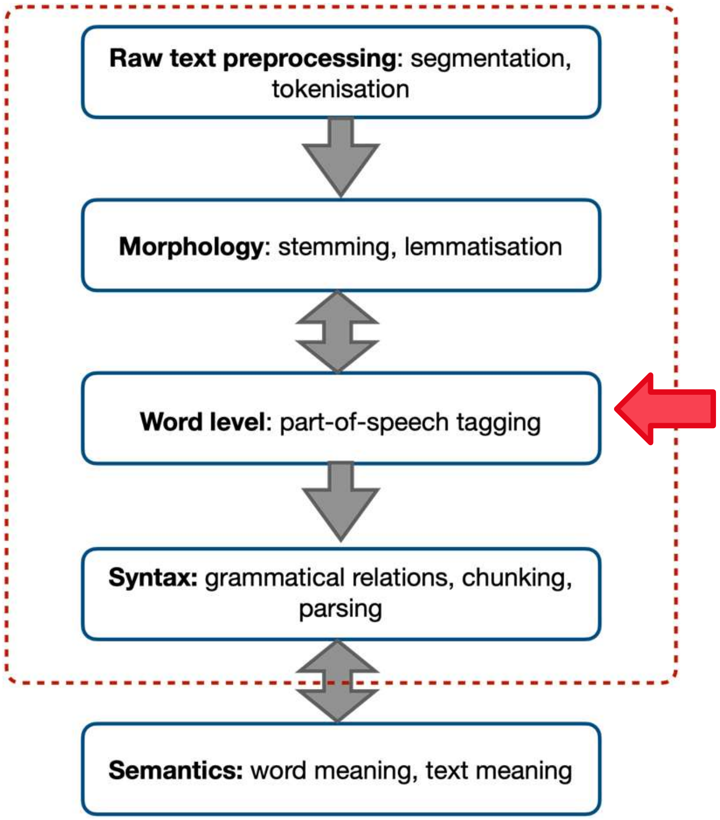
- Some words are part of **larger groups** that should be treated as a **single unit** (Albert Einstein, German Empire)
- Certain (groups of) words fulfil **certain roles**:
  - Albert Einstein is a person's name (*who?*)
  - German Empire is a location (*where?*)
  - 14 March 1879 is a date (*when?*)

They can answer certain questions but not others (just like "was born" cannot answer any of the above)



Challenges

- (Groups of) words occupy **specific positions** in a sentence and are **related to each other** in particular ways: e.g., "was born in" links the subject Albert Einstein to the location Ulm, and
- "was born on" links the subject to the time reference "14 March 1879"
- The **order of the words** matters and it's not random: e.g., "Was Einstein born Albert in 14 March …" would be much harder (if not impossible) to understand. this non-random order can be used by the NLP algorithms to learn about word groups, their functions, and their relations to each other.



Introduction to PoS tagging

Parts of speech (PoS)

- **Part of speech** is the general category of a word that is defined by its grammatical properties and relations to other word categories. **Part-of-speech tagging** is identification of parts of speech
- For example (only most frequent PoS are included below):

Part-of-speech	What it denotes	Examples
Nouns	Objects, people, animals, places, concepts, time references	*car*, *Einstein*, *dog*, *Paris*, *calculation*, *Friday*
Verbs	Actions, states, occurrences	*meet*, *stay*, *become*, *happen*
Adjectives	Qualities of objects, people, animals, places, concepts	*red car*, *clever man*, *big dog*, *beautiful city*, *fast calculation*
Adverbs	Qualities of actions, states, occurrences	*meet recently*, *stay longer*, *just become*, *happen suddenly*

Quiz

- What are the PoS for “Jabberwock”, “Bandersnatch”, and “frumious”?
- Why?

> “Beware the Jabberwock, my son!
> The jaws that bite, the claws that catch!
> Beware the Jubjub bird, and shun
> The frumious Bandersnatch!”

Quiz

- “Jabberwock” and “Bandersnatch” are **nouns** – e.g., note the use of an article “the” in front
- “frumious” is an **adjective** as it is used in front of a noun

> “Beware the Jabberwock, my son!
> The jaws that bite, the claws that catch!
> Beware the Jubjub bird, and shun
> The frumious Bandersnatch!”

Why this matters?

Who did what [to whom] [when] […]?

- **Who was born on March 14?** → (NOUN phrase) → Albert Einstein
- **What happened on March 14?** → (VERB phrase) → Albert Einstein was born
- **When was Albert Einstein born?** → (NOUN phrase) → March 14

Quiz: Why is this challenging?

- How many interpretations are there for “**We can fish**”?

Quiz: Why is this challenging?

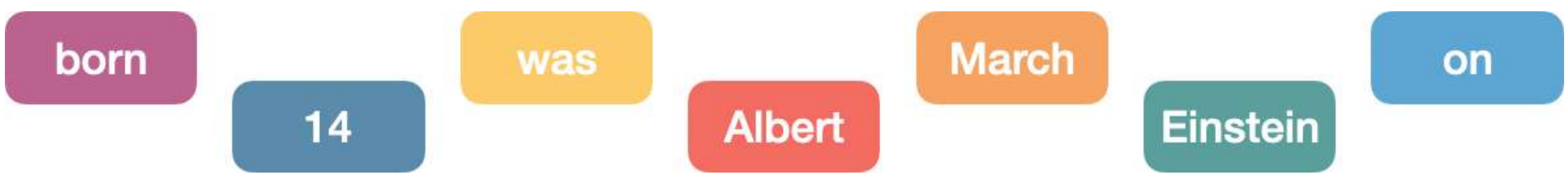
- How many interpretations are there for "**We can fish**"?
  - **We**$_{\text{PRONOUN}}$ **can**$_{\text{AUXILIARY}}$ **fish**$_{\text{VERB}}$ = "We know how to fish"
  - **We**$_{\text{PRONOUN}}$ **can**$_{\text{VERB}}$ **fish**$_{\text{NOUN}}$ = "We put fish in cans"

Sequence modelling and labelling in NLP

When the order matters

- However, texts are never just random collections of disconnected words
- Language has a clear inherent structure, with language rules governing the way words are put together in sentences and sentences are put together in larger units

Who did what [to whom] [when] […]?



When the order matters

- However, texts are never just random collections of disconnected words
- Language has a clear inherent structure, with language rules governing the way words are put together in sentences and sentences are put together in larger units

Who did what [to whom] [when] […]?

Sequence modelling:

in sequence modeling we learn patterns across sequences.
such as:

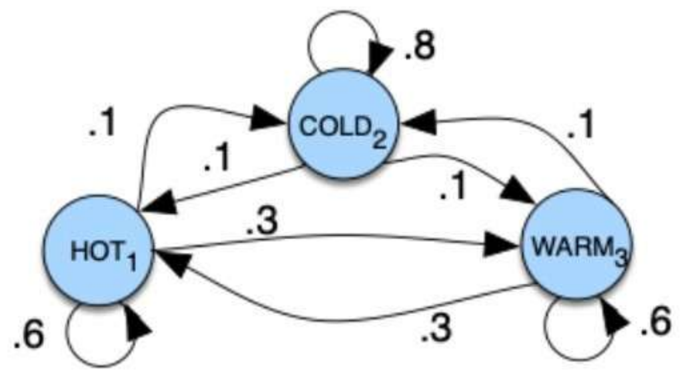
- What word is likely to come next?
- What part-of-speech (POS) fits after a given word?
- How does the structure of words and sentences form meaning?

Language = structured sequences.
And sequence modeling helps machines learn this structure.

https://web.stanford.edu/~jurafsky/slp3/8.pdf

Sequence modelling: Weather example

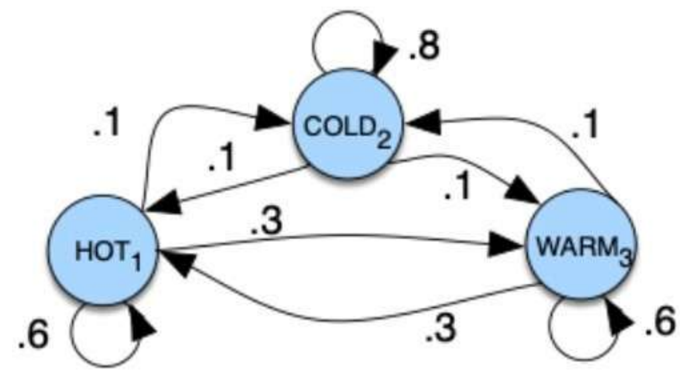
- Weather changes can be modelled **probabilistically**: e.g., it is more likely that a cold day is followed by another cold day than immediately by a hot one
- Let's represent this with a **directed graph**
- Vertices are **states**, and edges with associated probabilities are **transitions**



https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov chain for weather example

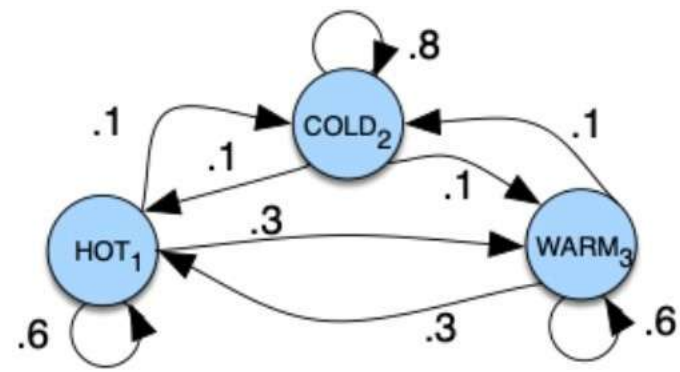
- The graph visualises a **Markov chain**
- A Markov chain is a model that defines the probabilities of sequences of random variables (states), which come from a predefined set
- E.g., $s_{weather}$ = [s_1=*hot*, s_2=*cold*, s_3=*warm*]



https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov chain for weather example

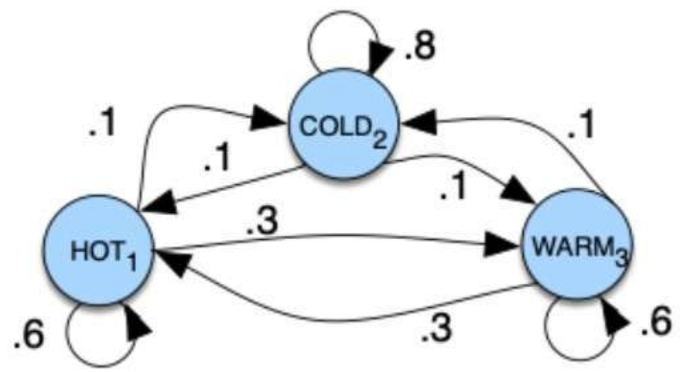
- The chain also defines **transition probabilities** a_i for each state *i*
- E.g., a_1 = [0.6, 0.1, 0.3]
- Interpretation:
    - ○ $P(s_1=hot \rightarrow s_1=hot) = 0.6$
    - ○ $P(s_1=hot \rightarrow s_2=cold) = 0.1$
    - ○ $P(s_1=hot \rightarrow s_3=warm) = 0.3$



https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov chain for weather example

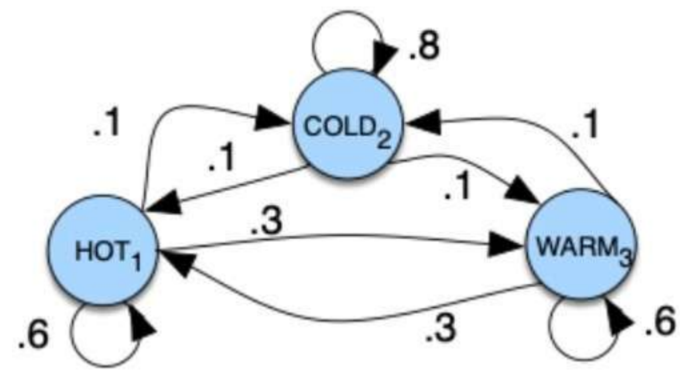
- Finally, let's define a **start distribution** π
- E.g., π = [0.1, 0.7, 0.2] (note that $\Sigma_i \pi_i = 1$)
- Interpretation:
  - P(starting at s_1=*hot*) = 0.1
  - P(starting at s_2=*cold*) = 0.7
  - P(starting at s_3=*warm*) = 0.2



https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov (independence) assumption

- **First-order** Markov chain: when predicting the future, the past does not matter, only the present
    - $P(s_i{=}a \mid s_1 \dots s_{i-1}) = P(s_i{=}a \mid s_{i-1})$
- **Interpretation**: to predict the weather for tomorrow, you only need to consider the weather for today
- The order of the model defines the length of the context



https://web.stanford.edu/~jurafsky/slp3/8.pdf

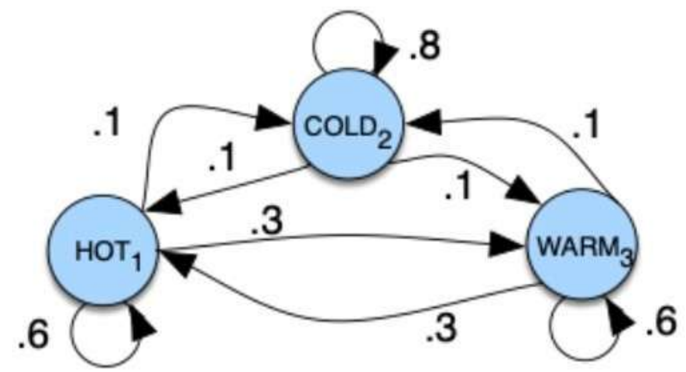
Markov model components

- $S = s_1\ s_2\ \ldots\ s_N$ – a set of *N* **states**
- $A = a_{11}\ a_{12}\ \ldots\ a_{N1}\ \ldots\ a_{NN}$ – a **transition probability matrix** *A* with a_{ij} representing the probability of moving from state s_i to state s_j:

$$\sum_j a_{ij} = 1\ \forall i$$

- $\pi = \pi_1, \pi_2, \ldots, \pi_N$ – an **initial probability distribution** over states

$$\sum_i \pi_i = 1$$

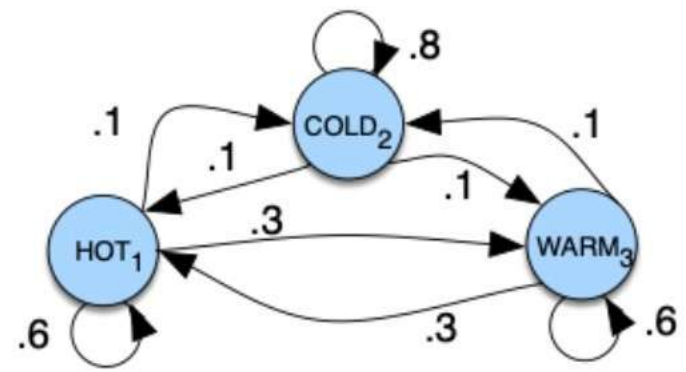


https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov model components

- π = [0.1, 0.7, 0.2]
- A:

	s_1 = hot	s_2 = cold	s_3 = warm
s_1 = hot	0.6	0.1	0.3
s_2 = cold	0.1	0.8	0.1
s_3 = warm	0.3	0.1	0.6

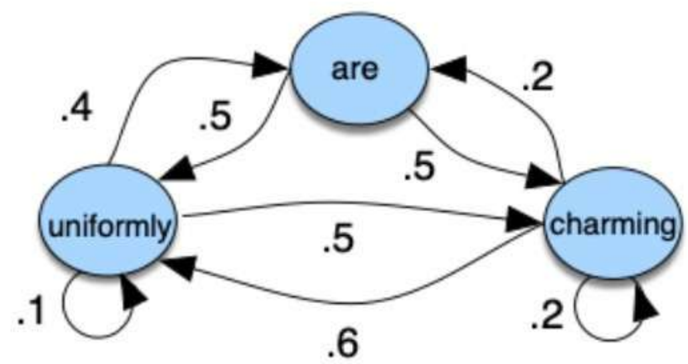


https://web.stanford.edu/~jurafsky/slp3/8.pdf

How does this apply to language?

- π = [0.1, 0.7, 0.2]
- A:

	uniformly	are	charming
uniformly	0.1	0.4	0.5
are	0.5	0.0	0.5
charming	0.6	0.2	0.2



https://web.stanford.edu/~jurafsky/slp3/8.pdf

Markov chain for language

- π = [0.1, **0.7**, 0.2]
- A:

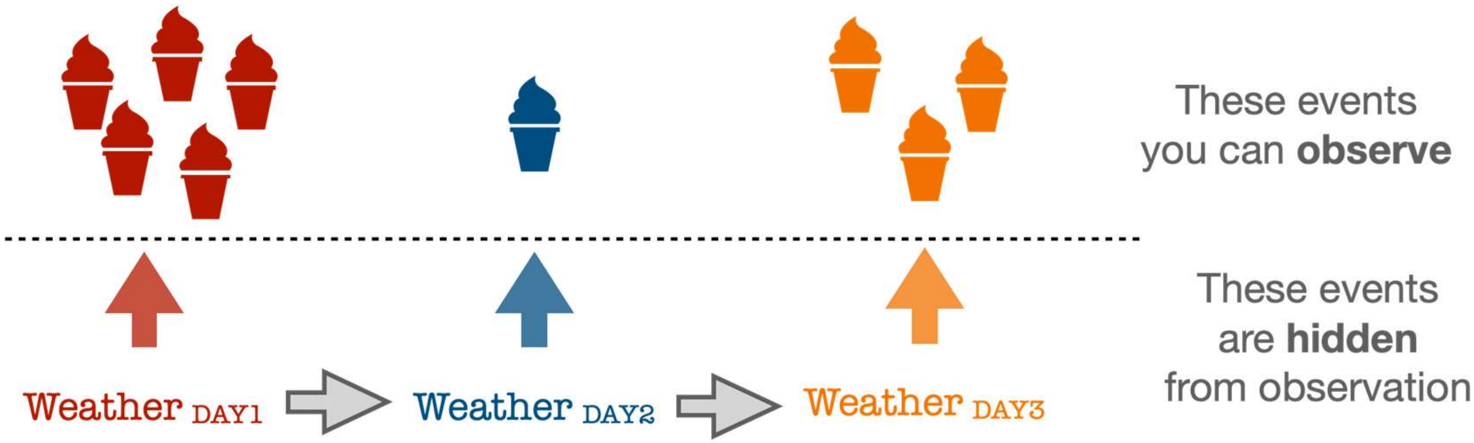
	uniformly	are	charming
uniformly	0.1	0.4	**0.5**
are	**0.5**	0.0	0.5
charming	0.6	0.2	0.2

P("are uniformly charming")?

= P(start with "are")
$\times$ P("are" $\rightarrow$ "uniformly")
$\times$ P("uniformly" $\rightarrow$ "charming")
= 0.7 $\times$ 0.5 $\times$ 0.5 = 0.175

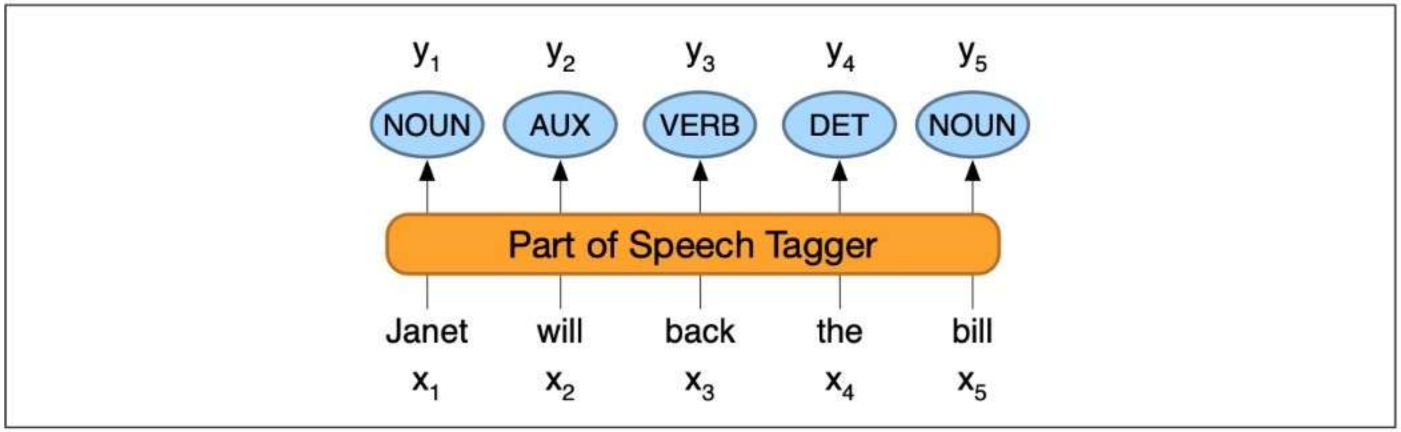
PoS tagging with HMMs

Hidden vs. observable events



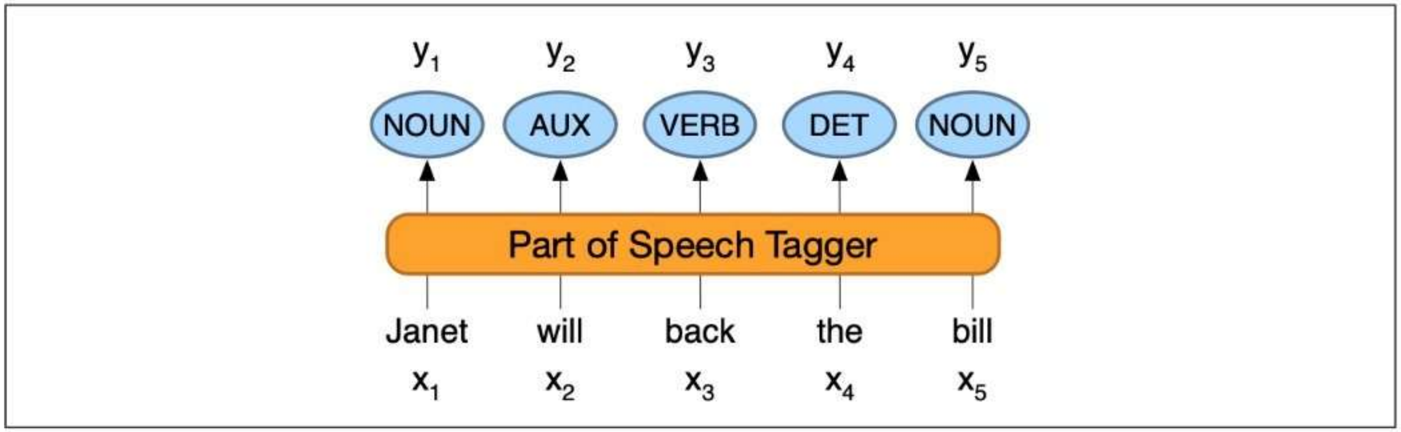
- Suppose you don't have access to historical data (e.g., on weather conditions)
- However, you recorded **observations** of some events that are dependent on weather conditions
- Can you identify what the **hidden** events (i.e., the weather conditions) were?

Part-of-speech (PoS) tagging



- You have access to **observations** (words)
- You are interested in detecting the underlying sequence of **hidden states** that produced these words (PoS)
- Words are not put together randomly → there are strong sequential effects

Part-of-speech (PoS) tagging



Goal: given the input (observable) sequence of words $x_1, \ldots, x_n$ and a tagset (collection of possible PoS tags), output the sequence $y_1, \ldots, y_n$, where each tag y_i corresponds to exactly one input x_i

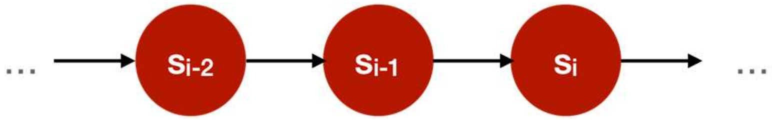
From a Markov Model

- $S = s_1\ s_2 \ldots s_N$ – a set of *N* **states**
- $A = a_{11}\ a_{12} \ldots a_{N1} \ldots a_{NN}$ – a **transition probability matrix** *A* with a_{ij} representing the probability of moving from state s_i to state s_j:

$$\sum_j a_{ij} = 1 \ \forall i$$

- $\pi = \pi_1, \pi_2, \ldots, \pi_N$ – an **initial probability distribution** over states

$$\sum_i \pi_i = 1$$

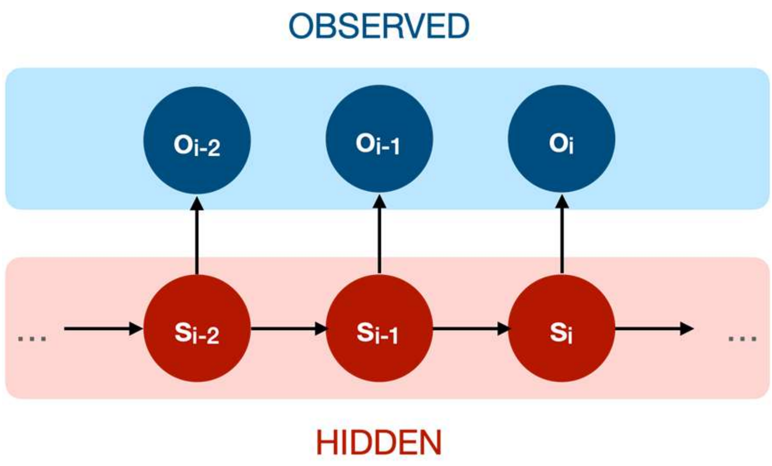


To a Hidden Markov Model (HMM)

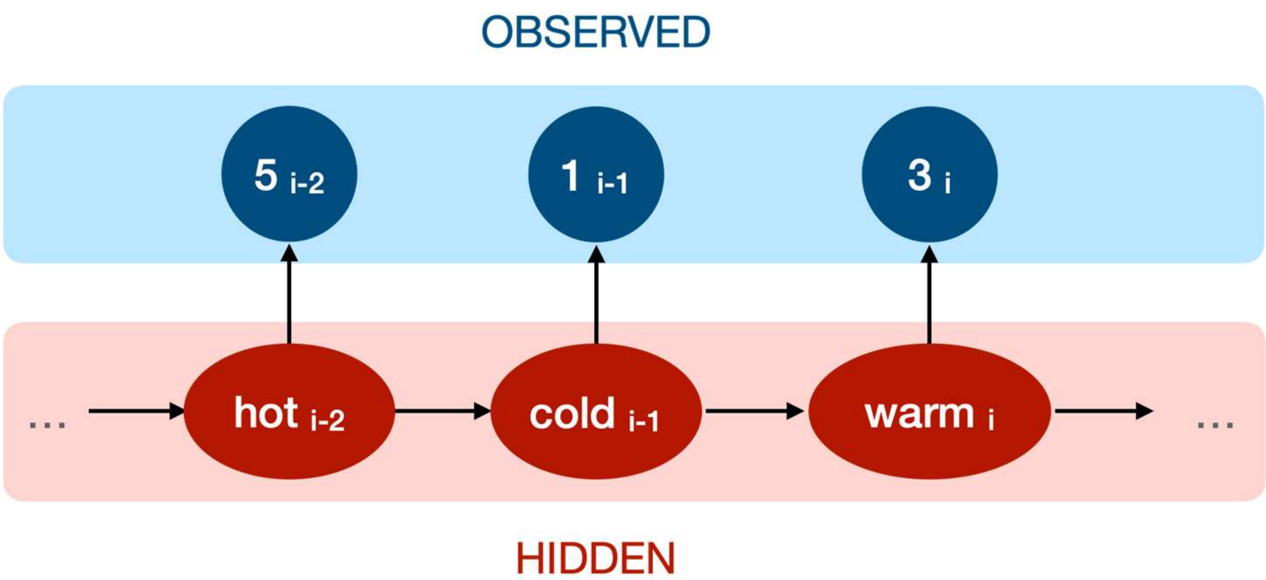
- $S=s_1\ s_2\ \ldots\ s_N$ – a set of *N* **states**
- $A= a_{11}\ a_{12}\ \ldots\ a_{N1}\ \ldots\ a_{NN}$ – a **transition probability matrix** *A*
- $\pi = \pi_1, \pi_2, \ldots, \pi_N$ – an **initial probability distribution** over states

AND

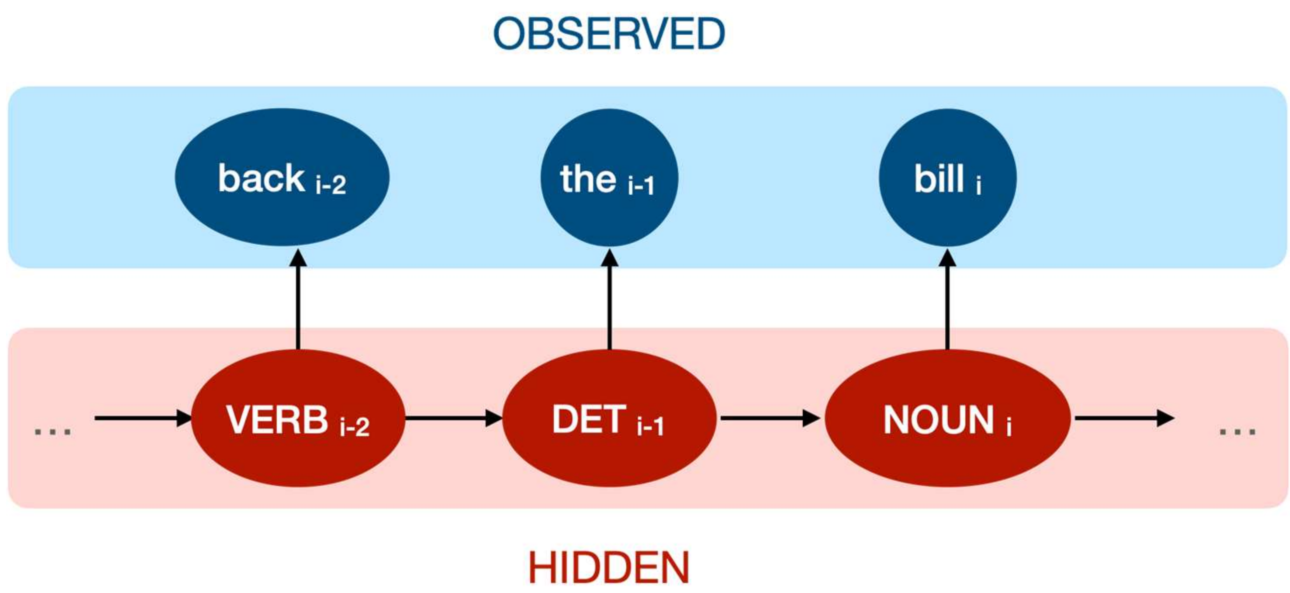
- $O=o_1\ o_2\ \ldots\ o_T$ – a sequence of *T* **observations**
- $B=b_i(o_t)$ – a sequence of **observation likelihoods** (**emission probabilities**), each expressing the probability of an observation o_t being generated from a state s_i



HMM: Ice cream example

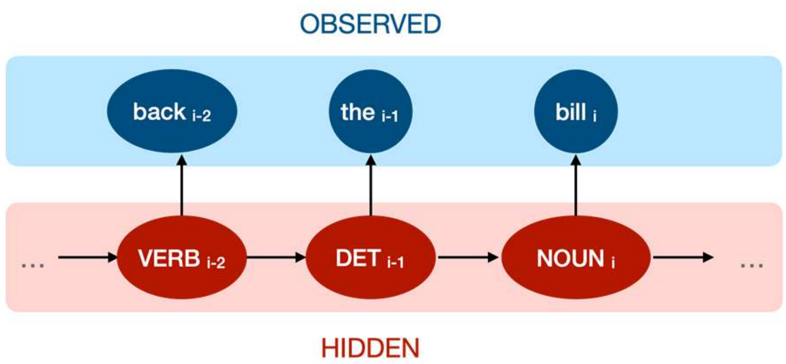


HMM: Part-of-speech example



What are the components?

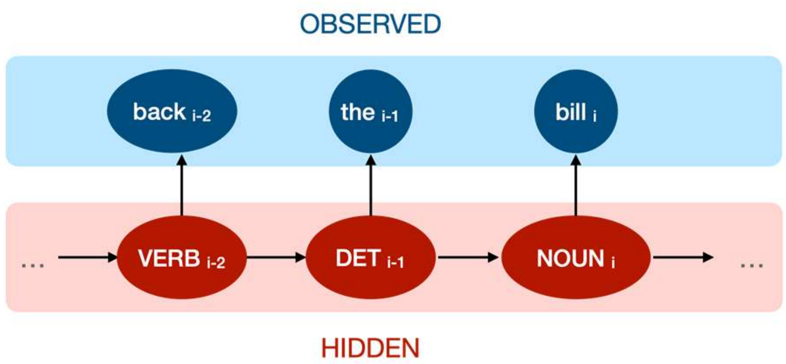
- $S = s_1\ s_2\ \ldots\ s_N$ – a set of *N* **states**



What are the components: States

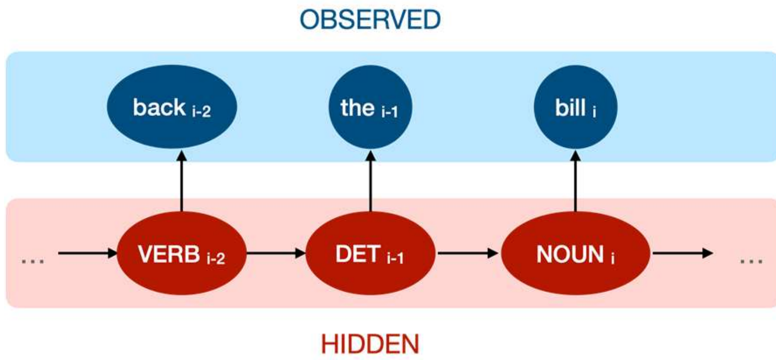
- $S = s_1\ s_2\ \ldots\ s_N$ – a set of *N* **states**

  S = [VERB, DET, NOUN]



What are the components?

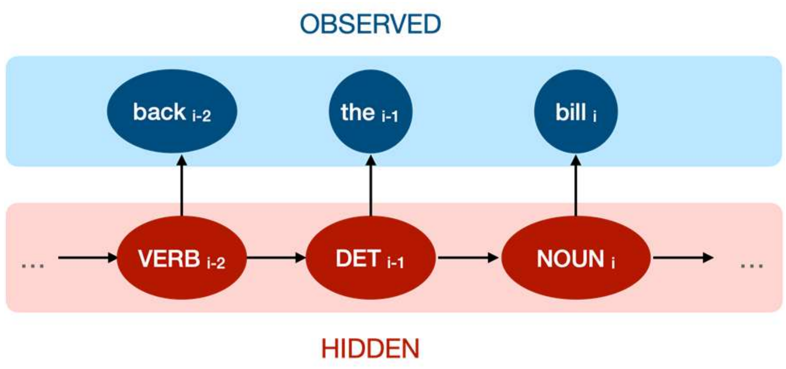
- A= a_{11} a_{12} … a_{N1} … a_{NN} – a **transition probability matrix** *A*



What are the components: Transitions

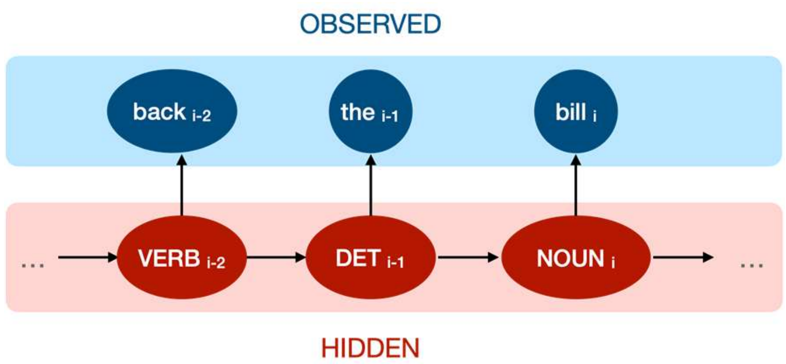
- $A = a_{11}\ a_{12} \ldots a_{N1} \ldots a_{NN}$ – a **transition probability matrix** *A*

	VERB	DET	NOUN
VERB	P(VERB → VERB)	P(VERB → DET)	…
DET	P(DET → VERB)	…	…
NOUN	…	…	…



What are the components?

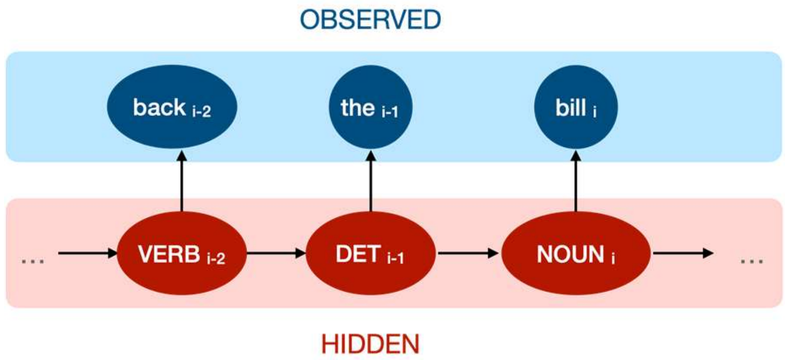
- $\pi = \pi_1, \pi_2, \ldots, \pi_N$ – an **initial probability distribution** over states



What are the components: Initial probability

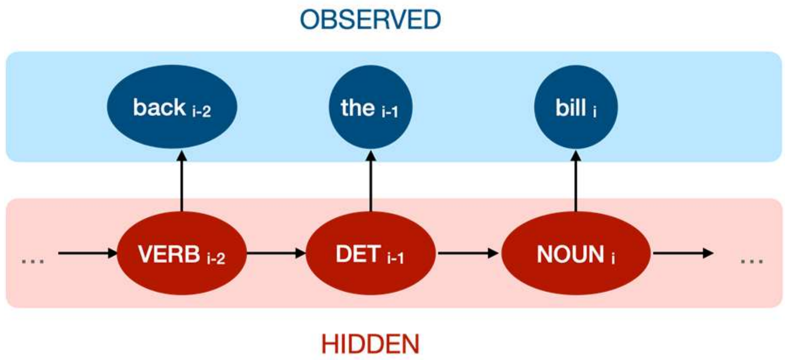
- $\pi = \pi_1, \pi_2, \ldots, \pi_N$ – an **initial probability distribution** over states

VERB	DET	NOUN
P(Start with VERB)	P(Start with DET)	P(Start with NOUN)



What are the components?

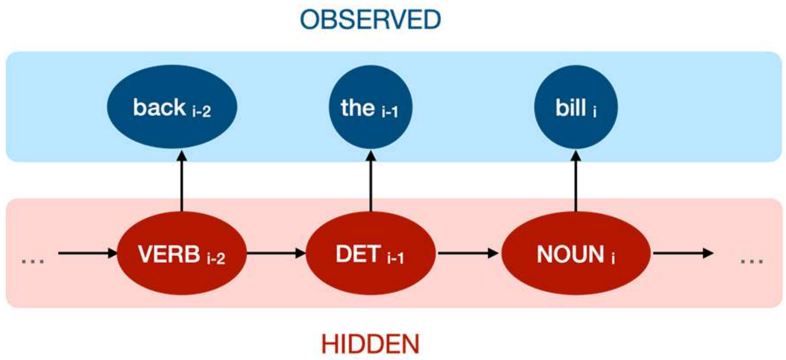
- $O = o_1 \, o_2 \, \ldots \, o_T$ – a sequence of T **observations**



What are the components: Observations

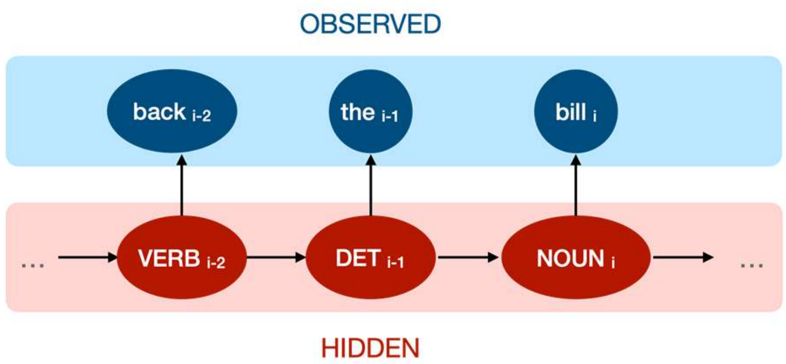
- $O = o_1\ o_2 \ldots o_T$ – a sequence of T **observations**

  O = [back, the, bill]



What are the components?

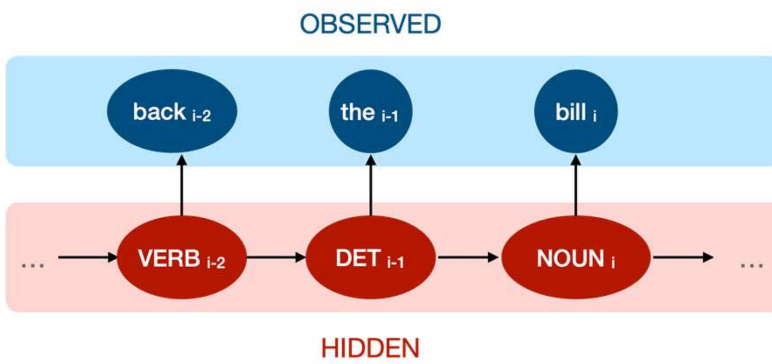
- $B = b_i(o_t)$ – a sequence of **observation likelihoods** (**emission probabilities**), each expressing the probability of an observation o_t being generated from a state s_i



What are the components: Emissions

- $B = b_i(o_t)$ – a sequence of **observation likelihoods** (**emission probabilities**), each expressing the probability of an observation o_t being generated from a state s_i

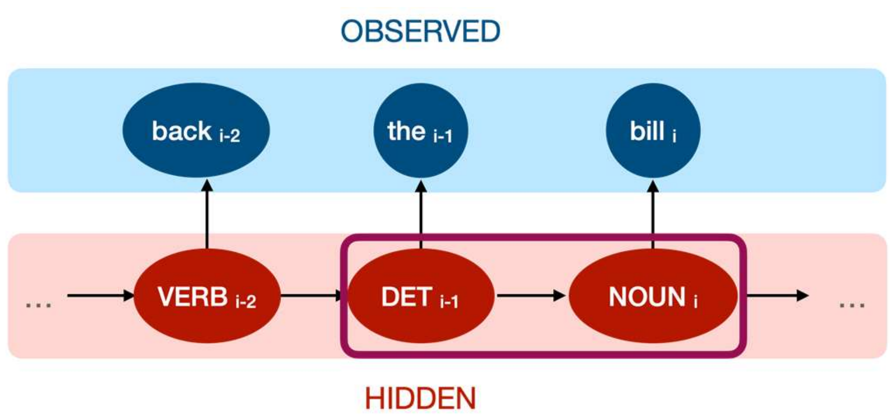
	back	the	bill
VERB	P(VERB → "back")	P(VERB → "the")	…
DET	P(DET → "back")	…	…
NOUN	…	…	…



Markov Model Assumption

First-order Markov Model assumption: The likelihood of transitioning to the current state s_i (e.g., tag_i=NOUN) **depends only on one previous state** (e.g., tag_{i-1}=DET) and no further history

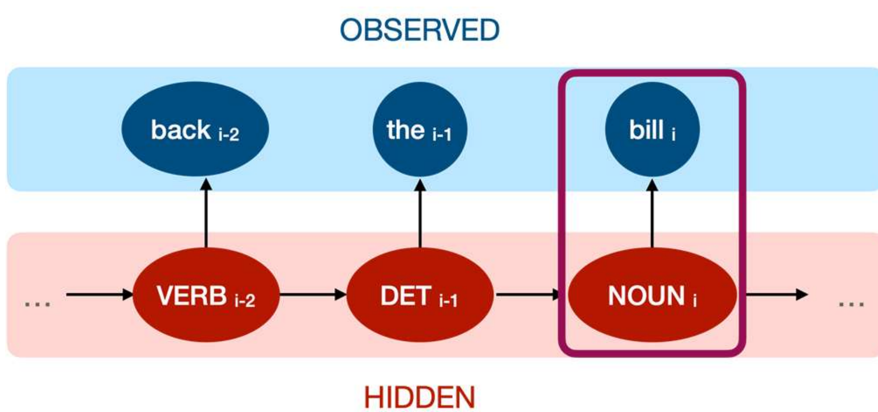
$$P(s_i \mid s_1 \ldots s_{i-1}) = P(s_i \mid s_{i-1})$$



+Output Independence Assumption

The probability of an output observation (e.g., $word_i$="bill") **depends only on the state** s_i that produced this observation (e.g., tag_i=NOUN) and not on any preceding states or any other observations

$$P(o_i \mid s_1, \ldots, s_i, \ldots, s_T, o_1, \ldots o_i, \ldots, o_T) = P(o_i \mid s_i)$$



Decoding

- **Overall goal:** determine the sequence of hidden variables corresponding to the sequence of observations (e.g., tags corresponding to words)
- **Formally**: given
    - the **transition matrix** A
    - the **emission matrix** B
    - the sequence of **observations** (words) $O = w_1 w_2 \ldots w_T$

  find the most probable sequence of **states** (tags) $S = t_1 t_2 \ldots t_T$

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\operatorname{argmax}} P(t_1 \ldots t_n | w_1 \ldots w_n)$$

Challenge

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} P(t_1 \dots t_n | w_1 \dots w_n)$$

- This means detecting that **P(VERB DET NOUN | “back the bill”)** is higher (e.g., more frequent in the data) than all other possible alternatives
- This is hard to estimate directly from the data (think of all possible tag sequences for all possible word combinations)

Solution

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} P(t_1 \dots t_n | w_1 \dots w_n)$$

- **Use Bayes rule** to flip the conditions:

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} \frac{P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)}{P(w_1 \dots w_n)}$$

- **Drop the denominator** (why can you do that?)

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)$$

Solution

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\operatorname{argmax}} P(w_1 \ldots w_n | t_1 \ldots t_n) P(t_1 \ldots t_n)$$

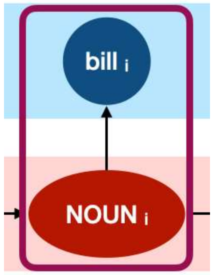
- This formula still relies on dependencies based on long sequences of tags and words. Can you simplify it further?

Independence assumptions

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\operatorname{argmax}} P(w_1 \ldots w_n | t_1 \ldots t_n) P(t_1 \ldots t_n)$$

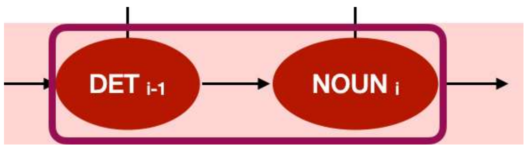
- This formula still relies on dependencies based on long sequences of tags and words. Apply **HMM independence assumptions**: for outputs

$$P(w_1 \ldots w_n | t_1 \ldots t_n) \approx \prod_{i=1}^{n} P(w_i | t_i)$$



and for tag sequences (hidden states):

$$P(t_1 \ldots t_n) \approx \prod_{i=1}^{n} P(t_i | t_{i-1})$$



To summarise

- We have started with:

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} P(t_1 \dots t_n | w_1 \dots w_n)$$

- And now we've ended up with:

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\operatorname{argmax}} P(t_1 \dots t_n | w_1 \dots w_n) \approx \underset{t_1 \dots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}}$$

- I.e., we can directly plug in the values from our **transition matrix A** and **emission matrix B**

Transition probabilities

- Estimate

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

- For example, suppose tag DET occurs 1,000 times and in 850 of these cases it is followed by NOUN. What is P(NOUN | DET)?

Transition probabilities

- Estimate

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1},t_i)}{C(t_{i-1})}$$

- For example, suppose tag DET occurs 1,000 times and in 850 of these cases it is followed by NOUN. What is P(NOUN | DET)?

- P(NOUN | DET) = *count* (DET NOUN) / *count* (DET) = 850 / 1000 = 0.85

Emission probabilities

- Estimate

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

- For example, suppose tag NOUN occurs 1,000 times and in 50 of these cases it is represented by a word “bill”. What is P(“bill” | NOUN)?

Emission probabilities

- Estimate

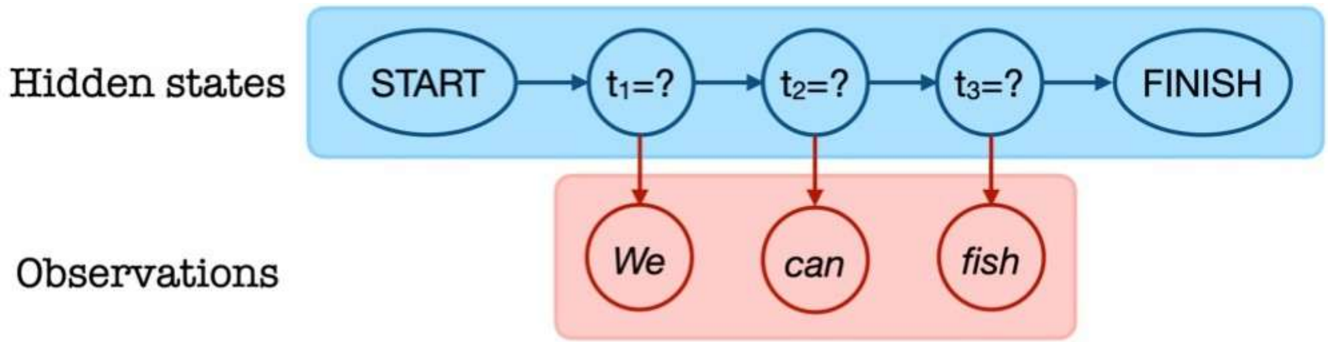
$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

- For example, suppose tag NOUN occurs 1,000 times and in 50 of these cases it is represented by a word “bill”. What is P(“bill” | NOUN)?
- P(“bill” | NOUN) = *count* (NOUN, “bill”) / *count* (NOUN) = 50 / 1000 = 0.05

Worked example

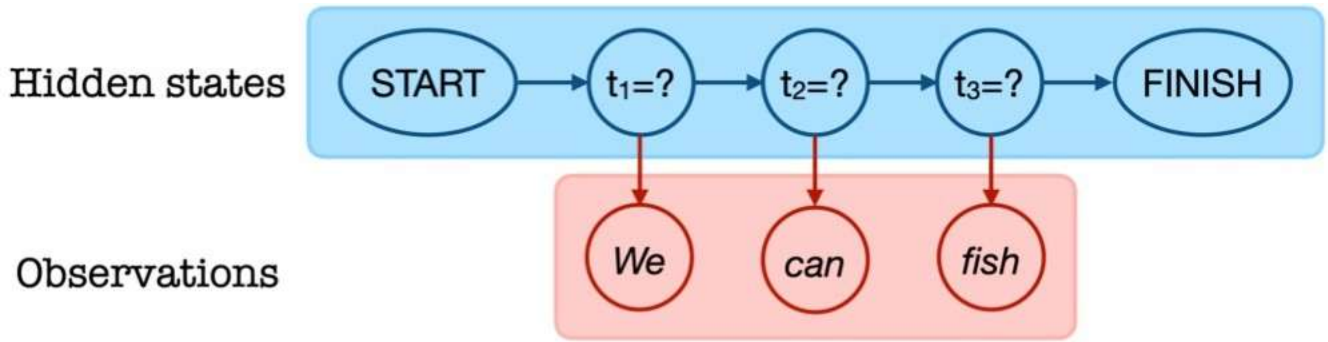
- How many interpretations are there for “**We can fish**”?
    - **We**$_{\text{PRONOUN}}$ **can**$_{\text{AUXILIARY}}$ **fish**$_{\text{VERB}}$ = “We know how to fish”
    - **We**$_{\text{PRONOUN}}$ **can**$_{\text{VERB}}$ **fish**$_{\text{NOUN}}$ = “We put fish in cans”

PoS tagging task: Brute-force



- You need to consider all possible sequences of tags and select the highest scored one
- There are ***T*=3 observations** and ***N*=6 states**
- What's the complexity of this approach?

PoS tagging task: Brute-force



- You need to consider all possible sequences of tags and select the highest scored one
- There are ***T*=3 observations** and ***N*=4 states**
- The complexity is O(N^T). Here, **N^T = 64 combinations**. Note that this is a relatively short sequence…

Worked example: Transition probabilities

a_{ij}	s_1 = verb	s_2 = noun	s_3 = pronoun	s_4 = auxiliary	s_f = FINISH
s_0 = START	$a_{01} = 0.01$	$a_{02} = 0.10$	$\boxed{a_{03} = 0.60}$	$a_{04} = 0.29$	$a_{0f} = 0.00$
s_1 = verb	$a_{11} = 0.02$	$a_{12} = 0.63$	$a_{13} = 0.07$	$a_{14} = 0.13$	$a_{1f} = 0.15$
s_2 = noun	$a_{21} = 0.49$	$a_{22} = 0.20$	$a_{23} = 0.10$	$a_{24} = 0.01$	$\boxed{a_{2f} = 0.20}$
s_3 = pronoun	$a_{31} = 0.40$	$a_{32} = 0.05$	$a_{33} = 0.05$	$a_{34} = 0.40$	$a_{3f} = 0.10$
s_4 = auxiliary	$\boxed{a_{41} = 0.73}$	$a_{42} = 0.01$	$a_{43} = 0.15$	$a_{44} = 0.01$	$a_{4f} = 0.10$

- For example, **the most likely first word** is a pronoun (e.g., “we”)
- It is very likely that **a noun finishes a sentence** (e.g., “(a) fish” or “(a) lecture”)
- The most likely **transition from an auxiliary** (e.g., “can (do)” or “will (do)”) is to a verb (e.g., “do”)

Worked example: Emission probabilities

b_{it}	*fish*	*can*	*we*
s_1 = verb	$b_1(fish) = 0.89$	$b_1(can) = 0.10$	$\boxed{b_1(we) = 0.01}$
s_2 = noun	$b_2(fish) = 0.75$	$b_2(can) = 0.24$	$b_2(we) = 0.01$
s_3 = pronoun	$b_3(fish) = 0.00$	$b_3(can) = 0.00$	$\boxed{b_3(we) = 1.00}$
s_4 = auxiliary	$b_4(fish) = 0.00$	$\boxed{b_4(can) = 1.00}$	$b_4(we) = 0.00$

- For example, **it is very unlikely** that a verb generates the word “we”
- With absolute certainty **pronoun** generates “we” and **auxiliary** – “can”

Worked example:

1.Observations: O=(we,can,fish)

2.Hidden states: S={s1=verb, s2=noun, s3=pronoun, s4=auxiliary}

3.possible state sequences(without finish and start) : $4^3=64$

4.Determine which paths are non-zero:

But most sequences will have probability = 0, because some emissions = 0 (e.g., pronoun cannot emit can or fish). So we will only compute sequences with non-zero probability.

Worked example:

4.Determine which paths are non-zero:

1. **Word 1: "we"**
   - $P(\text{"we"}|\text{verb}) = 0.01$
   - $P(\text{"we"}|\text{noun}) = 0.01$
   - $P(\text{"we"}|\text{pronoun}) = 1.00$ **(Highest)**
   - $P(\text{"we"}|\text{auxiliary}) = 0.00$ (Impossible)

2. **Word 2: "can"**
   - $P(\text{"can"}|\text{verb}) = 0.10$
   - $P(\text{"can"}|\text{noun}) = 0.24$
   - $P(\text{"can"}|\text{pronoun}) = 0.00$ (Impossible)
   - $P(\text{"can"}|\text{auxiliary}) = 1.00$ **(Highest)**

3. **Word 3: "fish"**
   - $P(\text{"fish"}|\text{verb}) = 0.89$ **(Highest)**
   - $P(\text{"fish"}|\text{noun}) = 0.75$
   - $P(\text{"fish"}|\text{pronoun}) = 0.00$ (Impossible)
   - $P(\text{"fish"}|\text{auxiliary}) = 0.00$ (Impossible)

Valid sequences to check: Based on the non-zero emissions, the meaningful paths to check are combinations of:
Position 1: Pronoun (most likely), Verb, or Noun.
Position 2: Auxiliary (most likely), Verb, or Noun.
Position 3: Verb (most likely) or Noun.

Worked example:

$$\hat{t}_{1:n} = \underset{t_1 \ldots t_n}{\operatorname{argmax}} P(t_1 \ldots t_n | w_1 \ldots w_n) \approx \underset{t_1 \ldots t_n}{\operatorname{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i|t_i)}^{\text{emission}} \overbrace{P(t_i|t_{i-1})}^{\text{transition}}$$

5.Compute probability for each valid path:

$$P(\text{Path}) = P(\text{Start} \to S_1) \times P(\text{Word}_1|S_1) \times P(S_1 \to S_2) \times P(\text{Word}_2|S_2) \times P(S_2 \to S_3) \times P(\text{Word}_3|S_3) \times P(S_3 \to \text{Finish})$$

Path A: Pronoun $\to$ Auxiliary $\to$ Verb

(Corresponds to: "We" is a pronoun, "can" is an auxiliary, "fish" is a verb)

1. **Start $\to$ Pronoun (s_3):** $a_{03} = 0.60$
2. **Emission "we" from Pronoun:** $b_3(\text{"we"}) = 1.00$
3. **Pronoun $\to$ Auxiliary (s_4):** $a_{34} = 0.40$
4. **Emission "can" from Auxiliary:** $b_4(\text{"can"}) = 1.00$
5. **Auxiliary $\to$ Verb (s_1):** $a_{41} = 0.73$
6. **Emission "fish" from Verb:** $b_1(\text{"fish"}) = 0.89$
7. **Verb $\to$ Finish (s_f):** $a_{1f} = 0.15$

$$P(A) = 0.60 \times 1.00 \times 0.40 \times 1.00 \times 0.73 \times 0.89 \times 0.15$$

$$P(A) = 0.02339$$

Path B: Pronoun $\to$ Auxiliary $\to$ Noun

(Corresponds to: "We" is a pronoun, "can" is an auxiliary, "fish" is a noun)

1. **Start $\to$ Pronoun:** 0.60
2. **Emission "we":** 1.00
3. **Pronoun $\to$ Auxiliary:** 0.40
4. **Emission "can":** 1.00
5. **Auxiliary $\to$ Noun (s_2):** $a_{42} = 0.01$
6. **Emission "fish" from Noun:** $b_2(\text{"fish"}) = 0.75$
7. **Noun $\to$ Finish (s_f):** $a_{2f} = 0.20$

$$P(B) = 0.00036$$

Worked example:

Path C: Pronoun $\rightarrow$ Verb $\rightarrow$ Noun

(Corresponds to: "We" is a pronoun, "can" is a verb [like 'canning' fruit], "fish" is a noun)

1. **Start $\rightarrow$ Pronoun:** 0.60
2. **Emission "we":** 1.00
3. **Pronoun $\rightarrow$ Verb (s_1):** $a_{31} = 0.40$
4. **Emission "can" from Verb:** $b_1(\text{"can"}) = 0.10$
5. **Verb $\rightarrow$ Noun (s_2):** $a_{12} = 0.63$
6. **Emission "fish" from Noun:** $b_2(\text{"fish"}) = 0.75$
7. **Noun $\rightarrow$ Finish:** 0.20

$$P(C) = 0.002268$$

Conclusion

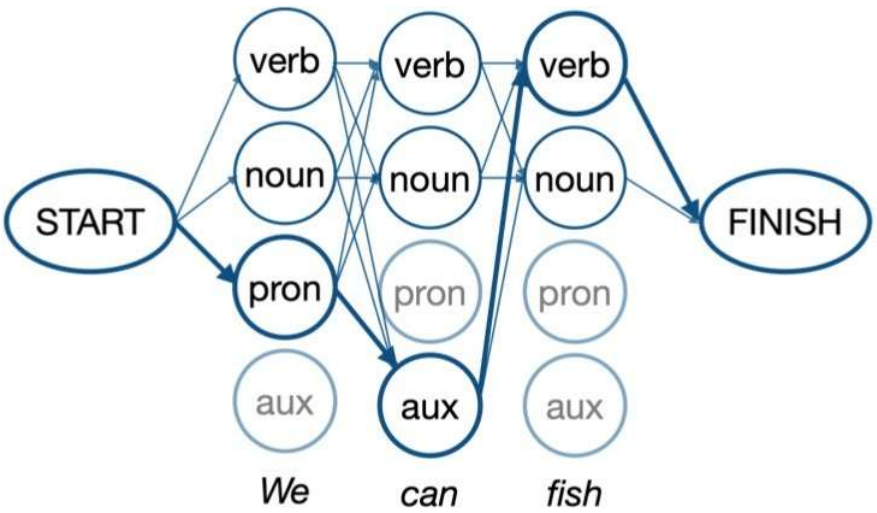
By comparing the probabilities calculated via brute force:

1. **Pronoun, Auxiliary, Verb: $\mathbf{0.02339}$**
2. **Pronoun, Verb, Noun:** 0.00227
3. **Pronoun, Auxiliary, Noun:** 0.00036

The best hidden state sequence is **Pronoun** $\rightarrow$ **Auxiliary** $\rightarrow$ **Verb**.

Dynamic programming: Viterbi

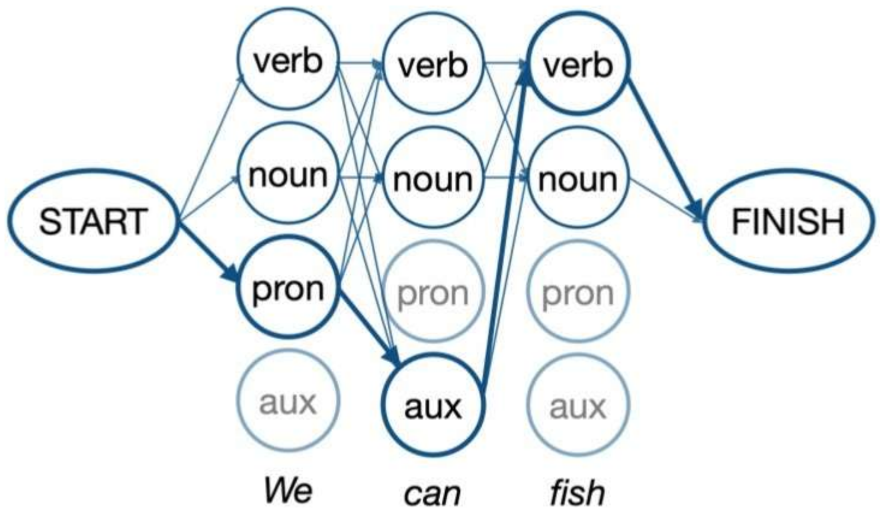
- Note that some sequences are implausible ⇒ you don't need to consider them at all
- **Viterbi:** Build a probability matrix (*lattice*) with one column per observation o_t and one row per state s_n



Viterbi dynamic programming approach

- At each step *t* and state *i*, we are going to only care about the most likely path from the previous point (*t-1*) that brought us here:

$$v_t(j) = max_{i=1}^{N} v_{t-1}(i) \cdot a_{ij} \cdot b_j(o_t)$$



where:
• vt-1(i) is the Viterbi path probability from the previous time step
• aij is the transition probability from the previous state si (tag) to the current state (tag) sj (Transition)
• bj(ot) is the state observation likelihood of the observation symbol (word) ot given the current state (tag) sj (Emission)

Dynamic programming approach

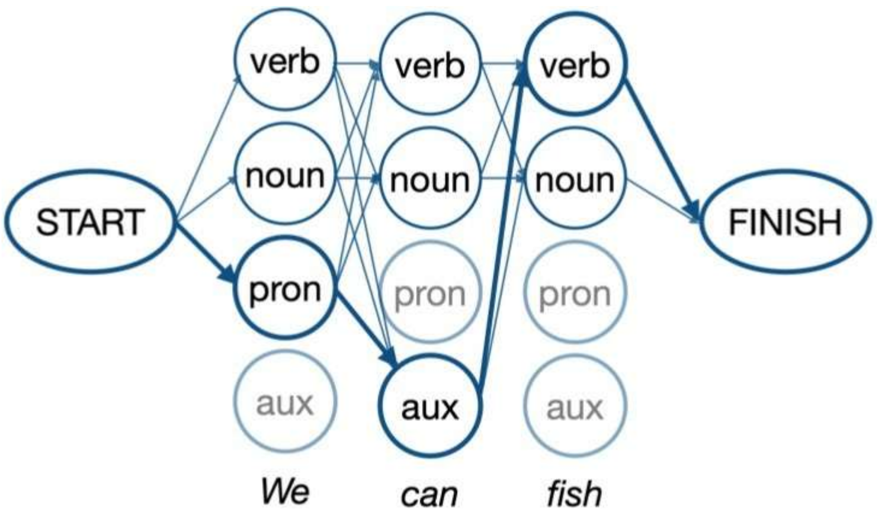
- Rely on the already estimated best case probabilities from time (*t-1*):

$$v_t(j) = max_{i=1}^{N} \boxed{v_{t-1}(i)} \cdot \boxed{a_{ij}} \cdot \boxed{b_j(o_t)}$$

Path probability for state *i* at *t-1*

Transition to current state *j*

Observation from the current state *j*



Observation at t=1: "We"

- Estimate:

$$v_{t=1}(j) = v_{t=0}(0) \cdot a_{0j} \cdot b_j(o_{t=1})$$

- Note that at this point every path leading to this observation starts at s_0=START (i.e., $v_{t=0}(0) = 1$)

- For each *j*, initialise $v_{t=0}(0) = 1$: you won't need to decide how you've arrived at *j*=NOUN or *j*=VERB at *t=1* since there is only a single START state at *t=0*

Worked example: Transition probabilities

a_{ij}	s_1 = verb	s_2 = noun	s_3 = pronoun	s_4 = auxiliary	s_f = FINISH
s_0 = START	$a_{01} = 0.01$	$a_{02} = 0.10$	$a_{03} = 0.60$	$a_{04} = 0.29$	$a_{0f} = 0.00$
s_1 = verb	$a_{11} = 0.02$	$a_{12} = 0.63$	$a_{13} = 0.07$	$a_{14} = 0.13$	$a_{1f} = 0.15$
s_2 = noun	$a_{21} = 0.49$	$a_{22} = 0.20$	$a_{23} = 0.10$	$a_{24} = 0.01$	$a_{2f} = 0.20$
s_3 = pronoun	$a_{31} = 0.40$	$a_{32} = 0.05$	$a_{33} = 0.05$	$a_{34} = 0.40$	$a_{3f} = 0.10$
s_4 = auxiliary	$a_{41} = 0.73$	$a_{42} = 0.01$	$a_{43} = 0.15$	$a_{44} = 0.01$	$a_{4f} = 0.10$

- Note that at this point every path leading to this observation starts at s_0=START

Worked example: Emission probabilities

b_{it}	***fish***	***can***	***we***
s_1 = verb	b_1(*fish*) = 0.89	b_1(*can*) = 0.10	b_1(*we*) = 0.01
s_2 = noun	b_2(*fish*) = 0.75	b_2(*can*) = 0.24	b_2(*we*) = 0.01
s_3 = pronoun	b_3(*fish*) = 0.00	b_3(*can*) = 0.00	b_3(*we*) = 1.00
s_4 = auxiliary	b_4(*fish*) = 0.00	b_4(*can*) = 1.00	b_4(*we*) = 0.00

- Take into account emission probabilities from each state
- Note that you don't need to consider s_0=START → s_4=auxiliary since probability of generating "we" from auxiliary equals 0

Worked example: Putting it all together

$$v_{t=1}(j=1) = v_{t=0}(0) \cdot a_{01} \cdot b_1(We) = 0.01 \cdot 0.01 = 0.0001$$ (”We” is a VERB)

$$v_{t=1}(j=2) = v_{t=0}(0) \cdot a_{02} \cdot b_2(We) = 0.10 \cdot 0.001 = 0.001$$ (“We” is a NOUN)

$$v_{t=1}(j=3) = v_{t=0}(0) \cdot a_{03} \cdot b_3(We) = 0.60 \cdot 1.00 = 0.60$$ (“We” is a PRONOUN)

Observation at t=2: “can”

- Estimate:

$$v_{t=2}(j) = max_{i=1}^{4} v_{t=1}(i) \cdot a_{ij} \cdot b_j(can)$$

- Now you need to consider all paths from time *t=1* and select the most likely one leading to each state at *t=2*

Observation at t=2: “can”

- Estimate: $$v_{t=2}(j) = max_{i=1}^{4} v_{t=1}(i) \cdot a_{ij} \cdot b_j(can)$$
- E.g., suppose you are at state **s_1=VERB** at *t=2* (i.e., “can” is a VERB)
- You need to **compare probabilities** of arriving at this state from the previous one at *t=1* if the previous one was s_2=NOUN (“We” was a NOUN), s_3=PRON (“We” was a PRON) and s_4=AUX (”We” was AUX)
- You then need to **select only the most probable** among such sequences (e.g., PRON VERB for ”We can”)
- Finally, you need to **consider all options** for states at *t*=2: s_2=NOUN (“can” is a NOUN), s_3=PRON (“can” is a PRON) and s_4=AUX (”can” is AUX)) and repeat the procedure

Worked example: Probabilities

a_{ij}	s_1 = verb	s_2 = noun	s_3 = pronoun	s_4 = auxiliary	s_f = FINISH
s_0 = START	$a_{01} = 0.01$	$a_{02} = 0.10$	$a_{03} = 0.60$	$a_{04} = 0.29$	$a_{0f} = 0.00$
s_1 = verb	$a_{11} = 0.02$	$a_{12} = 0.63$	$a_{13} = 0.07$	$a_{14} = 0.13$	$a_{1f} = 0.15$
s_2 = noun	$a_{21} = 0.49$	$a_{22} = 0.20$	$a_{23} = 0.10$	$a_{24} = 0.01$	$a_{2f} = 0.20$
s_3 = pronoun	$a_{31} = 0.40$	$a_{32} = 0.05$	$a_{33} = 0.05$	$a_{34} = 0.40$	$a_{3f} = 0.10$
s_4 = auxiliary	$a_{41} = 0.73$	$a_{42} = 0.01$	$a_{43} = 0.15$	$a_{44} = 0.01$	$a_{4f} = 0.10$

b_{it}	*fish*	*can*	*we*
s_1 = verb	$b_1(fish) = 0.89$	$b_1(can) = 0.10$	$b_1(we) = 0.01$
s_2 = noun	$b_2(fish) = 0.75$	$b_2(can) = 0.24$	$b_2(we) = 0.01$
s_3 = pronoun	$b_3(fish) = 0.00$	$b_3(can) = 0.00$	$b_3(we) = 1.00$
s_4 = auxiliary	$b_4(fish) = 0.00$	$b_4(can) = 1.00$	$b_4(we) = 0.00$

$$v_{t=2}(j) = max_{i=1}^{4} v_{t=1}(i) \cdot a_{ij} \cdot b_j(can)$$

where $v_{t=1}(1)$=0.0001, $v_{t=1}(2)$=0.001, $v_{t=1}(3)$=0.60, and $v_{t=1}(4)$=0.0

Worked example: for s_1=verb at t=2

a_{ij}	s_1 = verb	s_2 = noun	s_3 = pronoun	s_4 = auxiliary	s_f = FINISH
s_0 = START	a_{01} = 0.01	a_{02} = 0.10	a_{03} = 0.60	a_{04} = 0.29	a_{0f} = 0.00
s_1 = verb	a_{11} = 0.02	a_{12} = 0.63	a_{13} = 0.07	a_{14} = 0.13	a_{1f} = 0.15
s_2 = noun	a_{21} = 0.49	a_{22} = 0.20	a_{23} = 0.10	a_{24} = 0.01	a_{2f} = 0.20
s_3 = pronoun	a_{31} = 0.40	a_{32} = 0.05	a_{33} = 0.05	a_{34} = 0.40	a_{3f} = 0.10
s_4 = auxiliary	a_{41} = 0.73	a_{42} = 0.01	a_{43} = 0.15	a_{44} = 0.01	a_{4f} = 0.10

b_{it}	*fish*	*can*	*we*
s_1 = verb	b_1(*fish*) = 0.89	b_1(*can*) = 0.10	b_1(*we*) = 0.01
s_2 = noun	b_2(*fish*) = 0.75	b_2(*can*) = 0.24	b_2(*we*) = 0.01
s_3 = pronoun	b_3(*fish*) = 0.00	b_3(*can*) = 0.00	b_3(*we*) = 1.00
s_4 = auxiliary	b_4(*fish*) = 0.00	b_4(*can*) = 1.00	b_4(*we*) = 0.00

$$v_{t=2}(j) = max_{i=1}^{4} v_{t=1}(i) \cdot a_{ij} \cdot b_j(can)$$

where $\boxed{v_{t=1}(1)=0.0001}$ $\Rightarrow$ $v_{t=2}(1)=0.0001 \times 0.02 \times 0.1 = 2 \times 10^{-7}$

Worked example: Putting it all together

$$
\begin{aligned}
v_{t=2}(j=1) &= max_{i=1}^{4} v_{t=1}(i) \cdot a_{i1} \cdot b_1(can) = \\
&= max\{v_{t=1}(1) \cdot a_{11} \cdot b_1(can), v_{t=1}(2) \cdot a_{21} \cdot b_1(can), \\
&\qquad \boxed{v_{t=1}(3) \cdot a_{31} \cdot b_1(can)}, v_{t=1}(4) \cdot a_{41} \cdot b_1(can)\} = \\
&= max\{0.0001 \cdot 0.02 \cdot 0.10, 0.001 \cdot 0.49 \cdot 0.10, \boxed{0.60 \cdot 0.40 \cdot 0.10}, 0.00\} = \\
&= max\{2 \cdot 10^{-7}, 4.9 \cdot 10^{-5}, \boxed{2.4 \cdot 10^{-2}}, 0.00\} = \\
&= 0.024
\end{aligned}
$$

- State s_1 at t=2 will **only keep the information** about the path $s_3 \rightarrow s_1$ for the time t=1 → t=2 and **not any other transitions** since they are less likely

Where we are at

	t_1=*we*	t_2=*can*	t_3=*fish*
s_1=verb	0.01 * 0.01 = 0.0001 (s_0)	0.024 (s_3)	
s_2=noun	0.10 * 0.01 = 0.001 (s_0)		
s_3=pronoun	0.60 * 1.00 = 0.60 (s_0)		
s_4=auxiliary	0.29 * 0.00 = 0.00 (s_0)		

Final

Where we are at

	t_1=*we*	t_2=*can*	t_3=*fish*
s_1=verb	0.01 * 0.01 = 0.0001 (s_0)	0.024 (s_3)	0.155928 (s_4)
s_2=noun	0.10 * 0.01 = 0.001 (s_0)	0.0072 (s_3)	0.01134 (s_1)
s_3=pronoun	0.60 * 1.00 = 0.60 (s_0)	0.00 (any)	0.00 (any)
s_4=auxiliary	0.29 * 0.00 = 0.00 (s_0)	0.24 (s_3)	0.00 (any)

Final
0.023 (s_1)

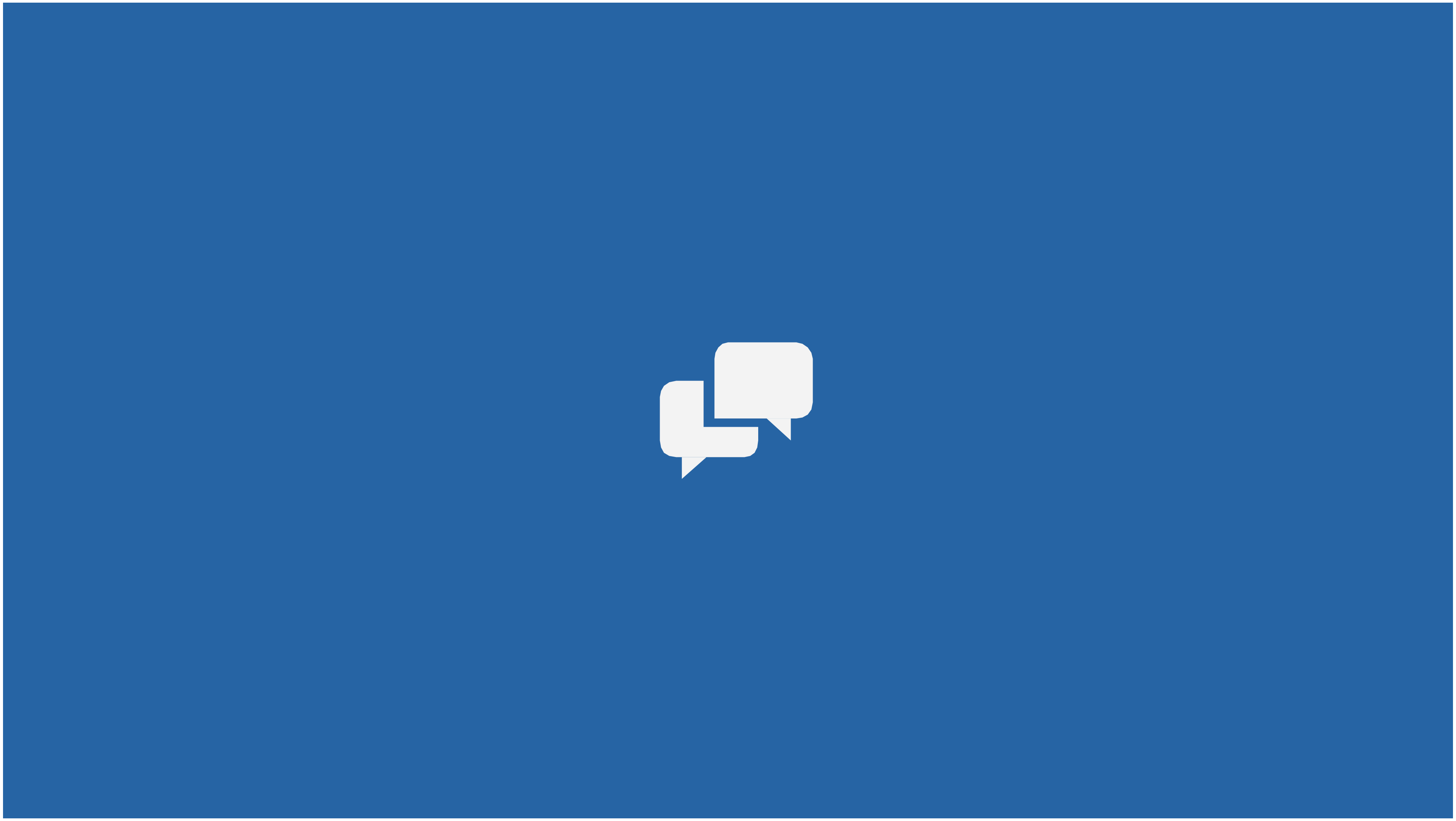
Check your understanding

- Follow the estimations to the final state: **Which interpretation wins?**
- **Answers** will be available in the handout

Summary

What you have learned this week

- The **task** of Information Extraction (IE) was introduced
- You've learned about such **fundamental NLP concepts** as *parts of speech* and *sequence modelling and labelling*
- You've learned about **key NLP techniques**: *PoS tagging*, using *Hidden Markov Models (HMMs)* and a dynamic programming *Viterbi algorithm*
- **Next time**: *syntactic processing* and *building end-to-end IE application*



Viterbi Example

O= The Fans Watch The Race

Transition	Dt	N	V
Start	0.8	0.2	0
Dt	0	0.9	0.1
N	0	0.5	0.5
V	0.5	0.5	0

Emission	the	Fans	Watch	Race
Dt	0.2	0	0	0
N	0	0.1	0.3	0.1
V	0	0.2	0.15	0.3